

PointProgrammerNet: Fixed-state additive programs for streaming point-cloud understanding

Dian Yu^{a,**}

^aUniversity of New South Wales, Sydney, NSW 2052, Australia

ABSTRACT

Point-cloud networks usually obtain context by revisiting points, building neighborhoods, or taking a global maximum. These operations work well offline, but a growing stream must then retain or reprocess its history. PointProgrammerNet writes each observation into three fixed-size fast-weight-programmer (FWP) matrices. Continuous normalized keys and outer-product values make the states additive, so independently processed chunks can be merged, removed, or exponentially decayed. The model retains three 128×170 float32 states (255 KiB), regardless of point count. Segmentation reads the states at a requested coordinate, whereas classification uses the states alone after discarding all points. We prove permutation invariance, chunk equivalence, fixed memory, and continuous dependence on new evidence. With XYZ input, PointProgrammerNet reaches 92.90% point accuracy, 82.39% instance mIoU, and 78.68% category mIoU on ShapeNetPart, and 86.59% accuracy on ModelNet40. At 8,192 accumulated points, updating only the new chunk is $5.54\times$ faster than rebuilding the state, with relative error below 8×10^{-7} . These results quantify the accuracy cost and streaming benefit of replacing point-history access with a bounded additive state.

© 2026 Elsevier Ltd. All rights reserved.

Keywords: point clouds ; streaming learning ; additive state ; fast-weight programmers ; segmentation ; classification

1. Introduction

A point cloud is an unordered set, but a sensor observes it over time. Processing within a frame should be independent of point order, and processing across frames should not require replaying the full history. PointNet Qi et al. (2017a) obtains a global descriptor by symmetric maximum pooling, while PointNet++ Qi et al. (2017b) builds multiscale metric neighborhoods. Both are effective offline. However, a maximum is not an additive state: fixed summaries cannot generally support deletion, independent chunk merging, and continuously weighted evidence by simple addition.

We study a stricter setting in which a point may be discarded immediately after it is written. Later computation accesses only fixed matrices, plus a query coordinate for

segmentation. A learned key distributes each point across state rows, and a second-order value records mass, position, learned content, and cross moments. Summing these per-point writes produces a fixed-size state.

Independent chunks can therefore be merged, a stored subset can be subtracted, and old evidence can be decayed. New points update the state without a pass over earlier observations. Segmentation reads a coordinate-conditioned field and stores no historical point features. Classification reads only state rows and discards the complete point list.

Our contributions are:

- a three-scale second-order FWP state whose memory does not grow with the number of observed points;
- additive point writes that allow independently encoded chunks to be merged without replaying their contents;
- a coordinate-conditioned segmentation head and a

^{**}Corresponding author
e-mail: dian.yu2@student.unsw.edu.au (Dian Yu)

state-only classification head, neither of which pools over decoded points;

- proofs and experiments covering additivity, state operations, accuracy, and update cost.

Our compact PointNet++ implementations are controlled baselines rather than claims of canonical reproduction. We use them to measure the accuracy cost of the fixed-state constraint.

2. Related Work

PointNet applies pointwise transforms followed by maximum pooling Qi et al. (2017a), and PointNet++ adds hierarchical metric neighborhoods Qi et al. (2017b). Deep Sets studies permutation-invariant functions built from sums Zaheer et al. (2017), while Set Transformer uses attention and inducing points Lee et al. (2019). Kernelized attention can be evaluated recurrently Katharopoulos et al. (2020), and fast-weight programmers express memory updates as additive outer products or delta corrections Schlag et al. (2021). PointProgrammerNet adapts this idea to geometry: its fixed states store spatial and learned second-order statistics and remain usable after the input points are released. We evaluate classification on ModelNet40 Wu et al. (2015) and part segmentation on ShapeNetPart Yi et al. (2016), which is derived from ShapeNet Chang et al. (2015).

3. Method

Let $X = \{\mathbf{x}_i\}_{i=1}^N$, $\mathbf{x}_i \in \mathbb{R}^3$. Reported models use XYZ; normals or other attributes may extend values without changing the algebra. We require

$$\mathbf{S}(A \uplus B) = \mathbf{S}(A) + \mathbf{S}(B). \quad (1)$$

Three fine, middle, and broad branches each use $R = 128$ rows and $D = 170$ columns. Their learnable radii start at 0.08, 0.20, 0.60. Figure 1 summarizes the shared additive encoder and the two task-specific readouts.

3.1. Continuous keys and second-order writes

Branch s has Sobol-initialized trainable centers $\mathbf{c}_{sj}$ and inverse bandwidth γ_s :

$$\ell_{sj}(\mathbf{x}) = -0.5\|\gamma_s(\mathbf{x} - \mathbf{c}_{sj})\|_2^2, \quad (2)$$

$$k_{sj}(\mathbf{x}) = \exp \ell_{sj}(\mathbf{x}) / \sum_{r=1}^R \exp \ell_{sr}(\mathbf{x}). \quad (3)$$

Thus every point continuously distributes unit mass across rows, without hard assignment or point comparison. A branch MLP gives $\mathbf{g}_s(\mathbf{x}) \in \mathbb{R}^{32}$ and

$$\mathbf{v}_s(\mathbf{x}) = [1, \mathbf{x}, \mathbf{g}_s, \text{vec}(\mathbf{x}\mathbf{g}_s^\top), x^2, y^2, z^2, xy, xz, yz, \mathbf{g}_s \odot \mathbf{g}_s] \in \mathbb{R}^{170}. \quad (4)$$

The FWP is

$$\mathbf{S}_s(X) = \sum_i \mathbf{k}_s(\mathbf{x}_i) \mathbf{v}_s(\mathbf{x}_i)^\top = \mathbf{K}_s^\top \mathbf{V}_s \in \mathbb{R}^{128 \times 170}. \quad (5)$$

The mass column normalizes each row. The remaining columns provide centroids, feature means and deviations, centered spatial moments, and position–feature cross moments.

3.2. Segmentation readout

At query $\mathbf{q}$, $\mathbf{a}_s(\mathbf{q}) = \mathbf{k}_s(\mathbf{q})^\top \mathbf{S}_s$. Density-normalized, query-centered moments pass through a branch MLP to $\mathbf{r}_s(\mathbf{q}) \in \mathbb{R}^{160}$. The head is

$$\mathbf{z}_{\text{seg}}(\mathbf{q}) = h_{\text{seg}}([\mathbf{r}_m; \mathbf{r}_f; \mathbf{r}_b; \phi(\mathbf{q}); \mathbf{e}_{\text{cat}}]). \quad (6)$$

Here $\phi(\mathbf{q}) \in \mathbb{R}^{32}$ encodes the query coordinate and $\mathbf{e}_{\text{cat}}$ is the ShapeNetPart category. The direct route $\mathbf{q} \mapsto \phi(\mathbf{q}) \mapsto h_{\text{seg}}$ preserves the query location while the FWP branches provide compressed multiscale context. This resembles a U-Net skip connection in purpose, although it carries a coordinate embedding rather than a matched-resolution feature map; we therefore call it a coordinate skip. The three state reads and the coordinate skip are concatenated once. Query points do not interact, and the decoder never takes a maximum over them. After writing, observed features can be deleted and only coordinates that will be queried need to remain. The same state can also be evaluated at a coordinate absent from the observed set, although we do not measure off-surface interpolation accuracy here.

3.3. Classification readout

For classification, each state row is converted to coordinate-free statistics and mapped to a token. Two learned soft weightings summarize the 128 rows of each branch, producing $\mathbf{h}_f, \mathbf{h}_m, \mathbf{h}_b \in \mathbb{R}^{256}$. The head processes these fixed rows rather than the input points:

$$\delta = \sigma(a) \tanh(W[\mathbf{h}_f; \mathbf{h}_b] + \mathbf{b}), \quad (7)$$

$$\mathbf{z}_{\text{cls}} = h_{\text{cls}}(\text{LN}(\mathbf{h}_m + \delta)). \quad (8)$$

The middle summary is the reference representation. A zero-initialized, bounded residual from the fine and broad summaries refines it without changing the initial function. The input points are deleted before this head runs. Only the middle radius uses a 25 $\times$ learning-rate multiplier.

3.4. Formal properties

Proposition 1 (Permutation invariance and additivity). For each branch, equation (5) is invariant to point order and $\mathbf{S}_s(\uplus_t X_t) = \sum_t \mathbf{S}_s(X_t)$.

Proof. Each point contributes one matrix. A finite sum is invariant to reordering and regrouping in real arithmetic; float differences arise only from accumulation order. $\square$

Corollary 1. Chunks merge by addition. If subset state $\mathbf{S}_s(Y)$ is available, $\mathbf{S}_s(X \setminus Y) = \mathbf{S}_s(X) - \mathbf{S}_s(Y)$. Exponential forgetting uses $\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \mathbf{S}_s(X_t)$, $0 \leq \rho \leq 1$.

Proposition 2 (Fixed memory and update). Retained memory is $O(\sum_s R_s D_s)$, independent of observed-point count. Encoding M new points costs $O(M \sum_s R_s D_s)$ for the outer products and is independent of history length.

Proof. Only fixed matrices persist; a new chunk produces the same matrix products regardless of earlier chunks. $\square$

Proposition 3 (Continuous evidence and queries). For evidence strength $t \in [0, 1]$, let

$$\mathbf{S}_s(t) = \mathbf{S}_s^{\text{old}} + t \mathbf{k}_s(\mathbf{x}) \mathbf{v}_s(\mathbf{x})^\top. \quad (9)$$

Classification logits and fixed-query segmentation logits are continuous, piecewise-smooth in t . Fixed-state segmentation logits are continuous, piecewise-smooth in $\mathbf{q}$.

Proof. The state is affine in t . RBFs, softmax, affine maps, positive-floor normalization, ReLU/GELU, and stabilized square roots are continuous; their composition is continuous and piecewise differentiable. $\square$

The state changes linearly with evidence strength. Logits and probabilities usually change nonlinearly but continuously, while a hard argmax label can switch at a decision boundary. This continuity follows from the model equations; calibration and temporal stability remain empirical questions.

3.5. Future frame-level delta rule

Let $\mathbf{K}_{s,t}$, $\mathbf{V}_{s,t}$ package all points of frame t . A future adaptive transition is

$$\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \eta_t \mathbf{K}_{s,t}^\top (\mathbf{V}_{s,t} - \mathbf{K}_{s,t} \mathbf{S}_{s,t-1}). \quad (10)$$

This follows fast-weight delta rules Schlag et al. (2021). Under within-frame permutation P , $\mathbf{K}' = P\mathbf{K}$, $\mathbf{V}' = P\mathbf{V}$, and the correction is unchanged because $P^\top P = I$. Thus points remain unordered within a frame while frame order enters the recurrence. This intentionally loses exact cross-frame additivity and is not used in our results.

4. Experiments

4.1. Protocol

We center each object and divide XYZ by its maximum absolute coordinate, using 1,024 points. ShapeNetPart has 13,972 training and 2,874 test shapes; ModelNet40 has 9,840 and 2,468. Models train 20 epochs with AdamW, base learning rate 3×10^{-3} , weight decay 10^{-4} , cosine decay, and task batch sizes 16 and 32. Classification uses 0.1 label smoothing. Values are mean $\pm$ population standard deviation for seeds 0,1,2. Density shift samples without replacement proportional to $\exp(2.5\mathbf{x}^\top \mathbf{d})$ for a fixed-seed random unit direction. The compact PointNet++ models and wider PointNet are controlled local baselines; the latter (442,838 parameters) is not exactly parameter-matched to PointProgrammerNet (529,426).

4.2. Accuracy

PointProgrammerNet is 0.51 points lower in point accuracy, 1.49 lower in instance mIoU, and 0.82 lower in category mIoU than lightweight PointNet++, while using 36.1% fewer parameters and no historical-point maximum.

The state-only classifier reaches 86.59% clean accuracy, 3.25 points below lightweight PointNet++ and 1.31 points below wider PointNet. Under the density shift, its accuracy drops by 4.12 points. Thus classification can be performed from mergeable states without retaining points, at a measurable accuracy cost.

4.3. Additivity and fixed state

Across 32 objects, states built from 2–32 ordered or shuffled chunks match one-shot construction with maximum relative error below 4.9×10^{-7} . The three states contain 65,280 float32 scalars (255 KiB). Fixed memory is not always smaller memory: raw XYZ reaches 255 KiB at 21,760 points, and a 1,024-point XYZ cloud is much smaller. The advantage is that retained memory remains bounded as the stream grows and stores learned geometric statistics rather than raw coordinates alone.

4.4. State operations

Append/merge, removal, and decay match direct constructions with mean relative errors 2.36×10^{-7} , 3.23×10^{-7} , and 1.92×10^{-7} ; all maxima remain below 8.9×10^{-7} . After 16 frames, the ordinary additive state has norm 14.57, whereas decay with $\rho = .9$ limits the norm to 7.45. This test checks the state algebra; it does not measure accuracy on a temporal task.

4.5. Streaming efficiency

On an RTX 3070 Ti Laptop GPU we time device-resident state construction for four streams, 256 new points per frame, 30 repetitions, and seven trials. Incremental latency remains near 1.5–1.6 ms. Rebuilding is slightly faster through 1,024 points due to fixed overhead. Speedup is 1.54 $\times$ at 2,048, 3.04 $\times$ at 4,096, and 5.54 $\times$ at 8,192 points; state error stays below 7.8×10^{-7} .

4.6. Controlled architecture audit

Five-epoch three-seed screens isolate design choices. Separated radii improve segmentation instance mIoU by 0.51 ± 0.18 points. Unified partition keys improve category mIoU by 0.82 ± 0.36 over mixed keys. Middle-reference residual classification improves clean and shifted accuracy by 2.67 ± 1.60 and 3.78 ± 1.71 over concatenation. The 25 $\times$ middle-radius rate has variable clean change (0.13 ± 1.03) but consistent shifted gain (0.91 ± 0.33), so we treat it as a robustness-oriented optimization choice.

Table 1. ShapeNetPart segmentation with XYZ input and 1,024 points. PointProgrammerNet does not pool across decoded query points. Values are mean $\pm$ population standard deviation over three seeds.

Model	Point acc. (%)	Ins. mIoU (%)	Cat. mIoU (%)	Params
Lightweight PointNet++	93.41 $\pm$ 0.05	83.88 $\pm$ 0.11	79.51 $\pm$ 0.24	573,106
PointProgrammerNet	92.90 $\pm$ 0.01	82.39 $\pm$ 0.12	78.68 $\pm$ 0.07	366,389

Table 2. ModelNet40 classification with 1,024 points. Density denotes evaluation under the fixed density-gradient shift. PointProgrammerNet classifies from fixed-size FWP states after discarding the input points.

Model	Clean (%)	Density (%)	Drop (pp)	Params
Lightweight PointNet++	89.84 $\pm$ 0.43	85.47 $\pm$ 0.88	4.38 $\pm$ 1.18	207,784
Wider PointNet	87.90 $\pm$ 0.27	87.55 $\pm$ 0.38	0.35 $\pm$ 0.12	442,838
PointProgrammerNet	86.59 $\pm$ 0.23	82.47 $\pm$ 0.93	4.12 $\pm$ 0.71	529,426

5. Discussion and Limitations

The intended setting is a growing point stream. A device can encode a frame, merge its state, and release the frame; separate workers can encode chunks independently; a segmentation system can retain only the coordinates it plans to query; and a classifier can update its evidence without replaying earlier points. None of these operations compares a new point with all previously decoded points.

The fixed state is also an information bottleneck. PointNet++ forms neighborhoods centered on observed points, and PointNet preserves channelwise extremes; PointProgrammerNet keeps only a finite set of learned moment tables. This explains part of the offline accuracy gap, especially for classification. The state also uses more memory than bare XYZ for small clouds, and segmentation still performs one state read for each requested coordinate.

Normals, color, or intensity can enter values while XYZ keys space. Normals might also define a product key, subject to orientation and bandwidth. Current evaluation covers one dataset per task, XYZ input, one timing GPU, and local controlled baselines. We have not measured unseen-query accuracy, temporal calibration, per-point class stability, or delta adaptation. Continuous logits do not imply monotonic improvement or a continuous hard label.

6. Conclusion

PointProgrammerNet summarizes a point cloud with fixed additive matrices. Continuous keys and second-order values support unordered writes, chunk merging, removal, decay, and updates whose cost does not depend on history length. Coordinate-conditioned segmentation and state-only classification operate without pooling over decoded points. Accuracy remains below the hierarchical baselines, but the model provides a compact and testable design for point-cloud inference from a bounded streaming state.

References

Chang, A.X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H., Xiao, J., Yi, L.,

Yu, F., 2015. Shapenet: An information-rich 3d model repository. arXiv:1512.03012 .
 Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F., 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention, in: ICML, pp. 5156–5165.
 Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., Teh, Y.W., 2019. Set transformer: A framework for attention-based permutation-invariant neural networks, in: ICML, pp. 3744–3753.
 Qi, C.R., Su, H., Mo, K., Guibas, L.J., 2017a. Pointnet: Deep learning on point sets for 3d classification and segmentation, in: CVPR, pp. 652–660.
 Qi, C.R., Yi, L., Su, H., Guibas, L.J., 2017b. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: NeurIPS.
 Schlag, I., Irie, K., Schmidhuber, J., 2021. Linear transformers are secretly fast weight programmers, in: ICML, pp. 9355–9366.
 Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., Xiao, J., 2015. 3d shapenets: A deep representation for volumetric shapes, in: CVPR, pp. 1912–1920.
 Yi, L., Kim, V.G., Ceylan, D., Shen, I.C., Yan, M., Su, H., Lu, C., Huang, Q., Sheffer, A., Guibas, L., 2016. A scalable active framework for region annotation in 3d shape collections. ACM Transactions on Graphics 35, 210:1–210:12.
 Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., Smola, A.J., 2017. Deep sets, in: NeurIPS.

Appendix A. Reproducibility

The accompanying code provides both task implementations, with 366,389 and 529,426 trainable parameters. Both expose `build_states` and `merge_states`; the task interfaces are `predict_from_states` and `classify_states`.

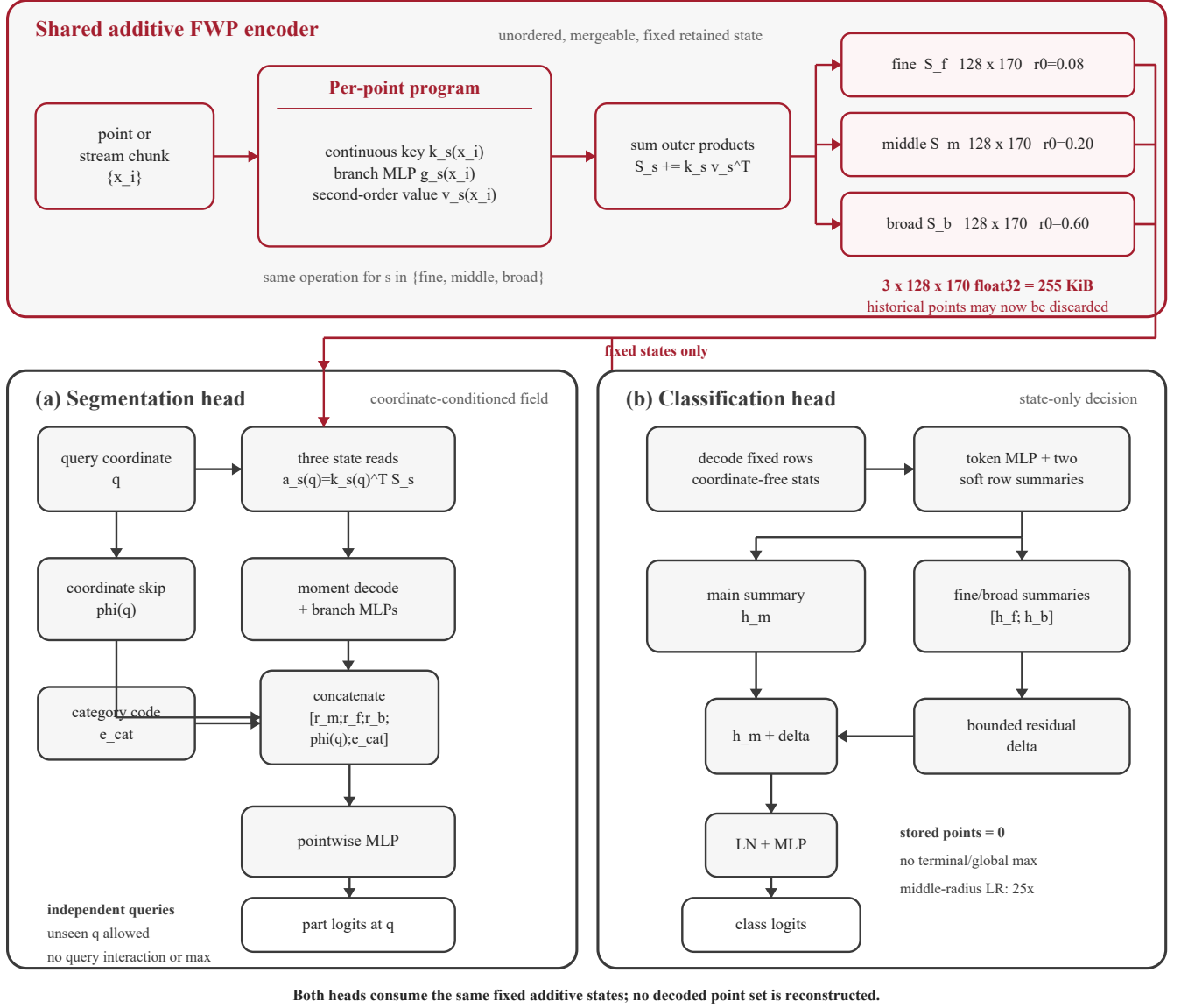


Fig. 1. PointProgrammerNet architecture. The shared encoder writes each observation into three equal-size additive FWP states, after which the historical point list can be discarded. For segmentation, the states are read at a query coordinate and combined with a direct coordinate skip. For classification, soft row summaries from the fine and broad states form a bounded residual around the middle-state summary. Neither head pools over decoded points.

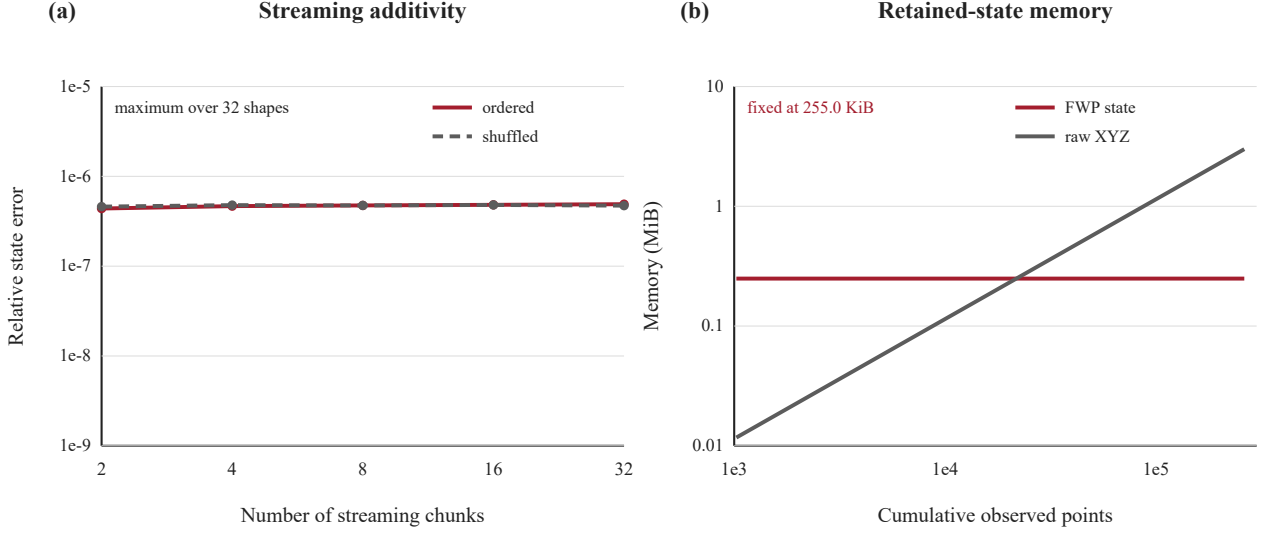


Fig. 2. Additive streaming and fixed retained memory. Ordered and shuffled chunk writes reproduce the single-batch FWP state up to floating-point accumulation error. The three 128×170 float32 states occupy 255 KiB regardless of stream length, whereas stored XYZ coordinates grow linearly with the number of observations.

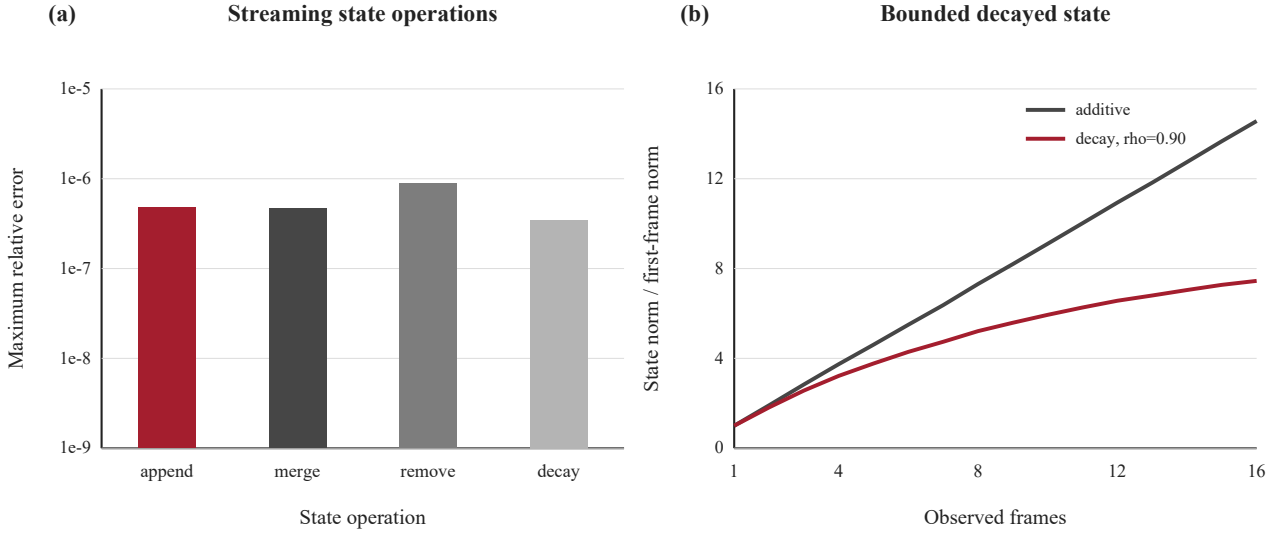


Fig. 3. Streaming state operations. (a) Maximum relative errors for append/merge, frame removal, and exponential decay, measured against independently reconstructed FWP states on 32 ShapeNetPart objects. (b) Ordinary addition increases the state magnitude with the number of frames, while exponential decay limits the influence of older frames without changing state size.

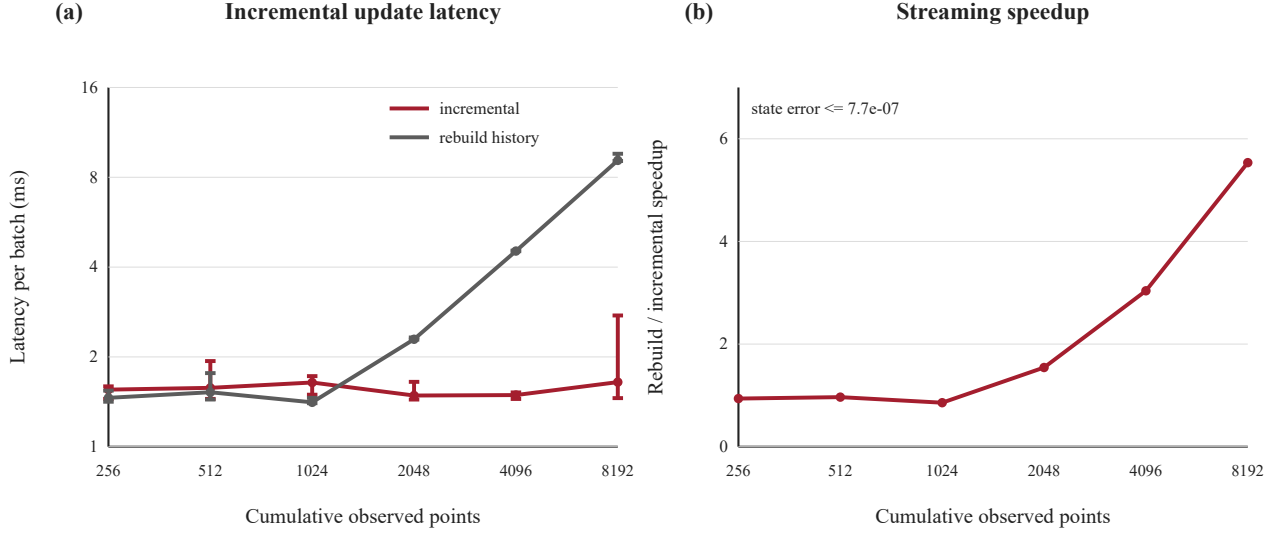


Fig. 4. FWP state-construction cost for device-resident point streams. (a) An incremental update processes one new 256-point frame, whereas reconstruction processes every accumulated point. Values are median wall-clock latency over seven interleaved trials of 30 repetitions for four streams; whiskers show the observed range. (b) The speedup grows with history length, while incremental and reconstructed states remain numerically equivalent.

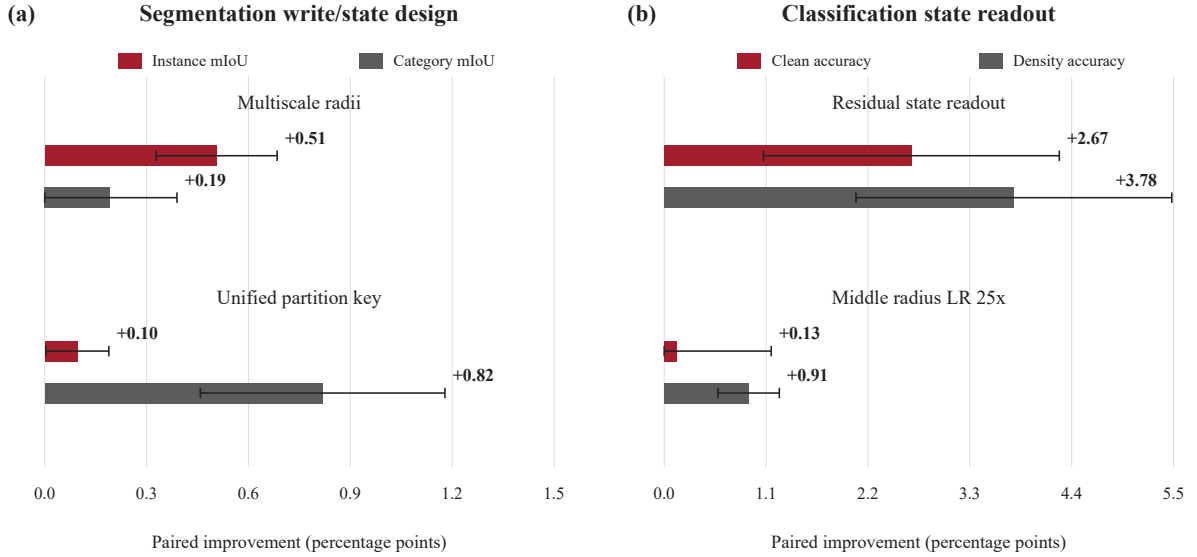


Fig. 5. Three-seed screening ablations. (a) For segmentation, separated initial radii improve equal partition states, and a common partition-key formulation improves the multiscale mixed-key control. (b) For state-only classification, the middle-reference residual improves clean and density-shift accuracy over direct concatenation; a higher learning rate for the middle radius further improves shifted accuracy. Bars show paired mean changes and whiskers show population standard deviations. Each screening run uses five epochs.